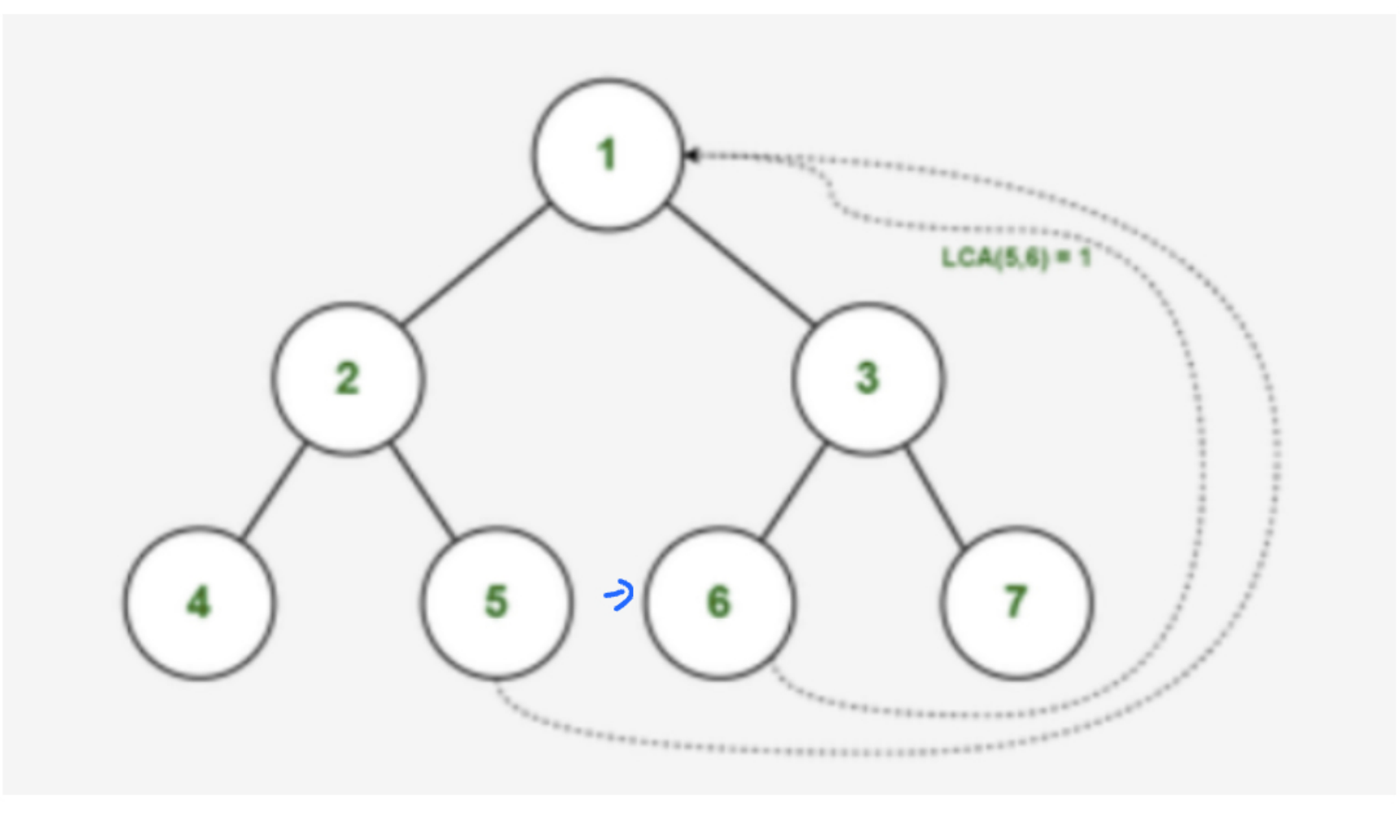
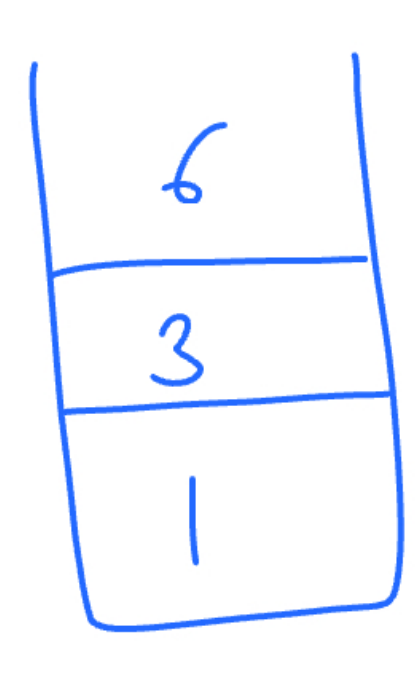
L C A.

(lowest) Common Ansrestor.



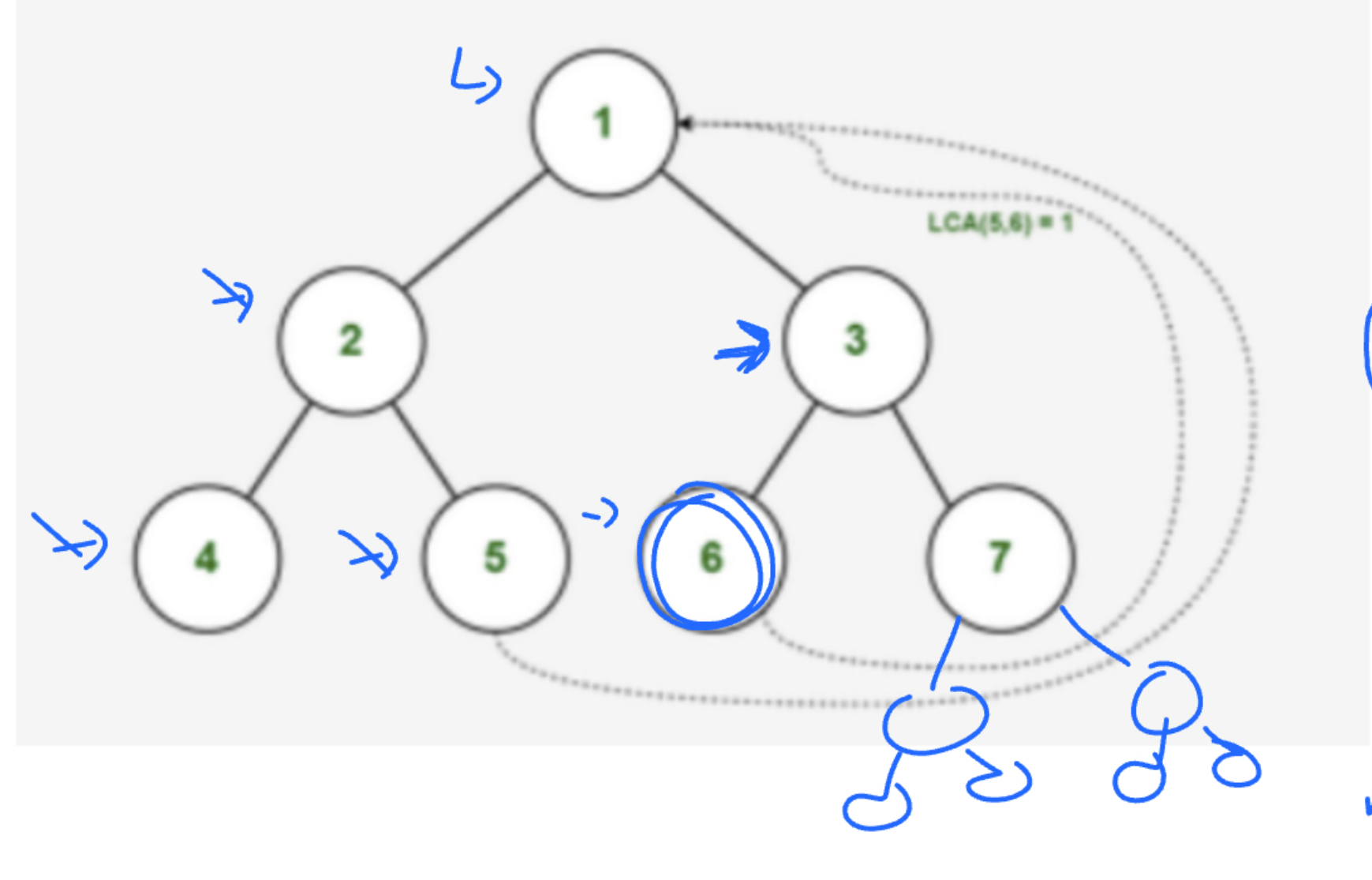
1
2
3
4
5
6
7
LCA(5,6) = 1



n1 = 6   n2 = 7.

→ 6: [1, 3, 6] ←

→ 7: [1, 3, 7] ←



1
2
3
4
5
6
7
LCA(5,6) = 1

curr: 1, ~~2, 4, 5~~, 3, 6

res: [1, 3, 6].

n1 = 6

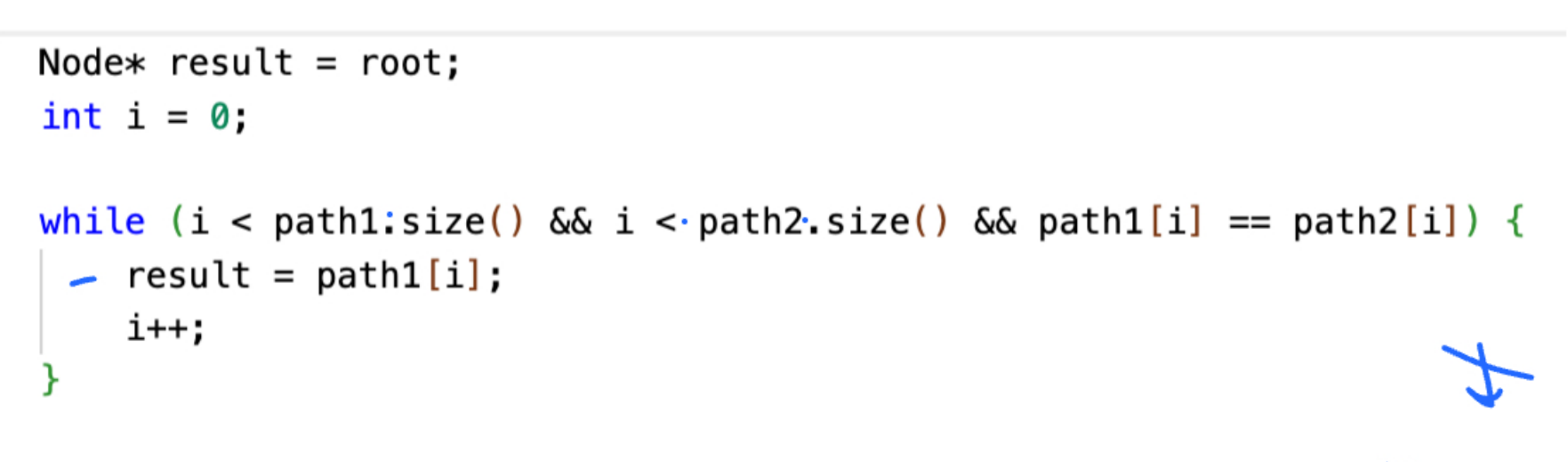
n2 = 3.

6: [1, 3, 6].

3: [1, 3].

val = 6.

res = ~~1~~ 3



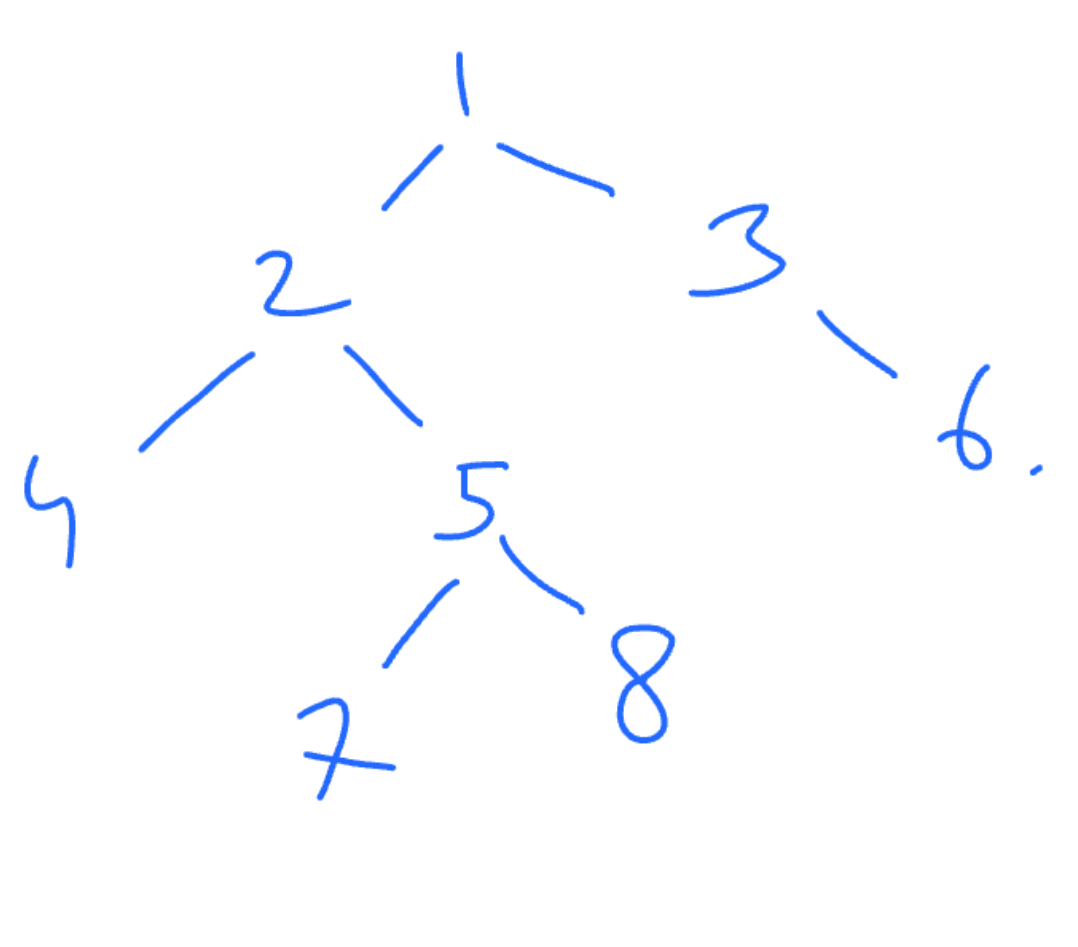
```
Node* result = root;
int i = 0;

while (i < path1.size() && i < path2.size() && path1[i] == path2[i]) {
    result = path1[i];
    i++;
}
```

6: [1, 3, 6]

7: [1, 3, 7]

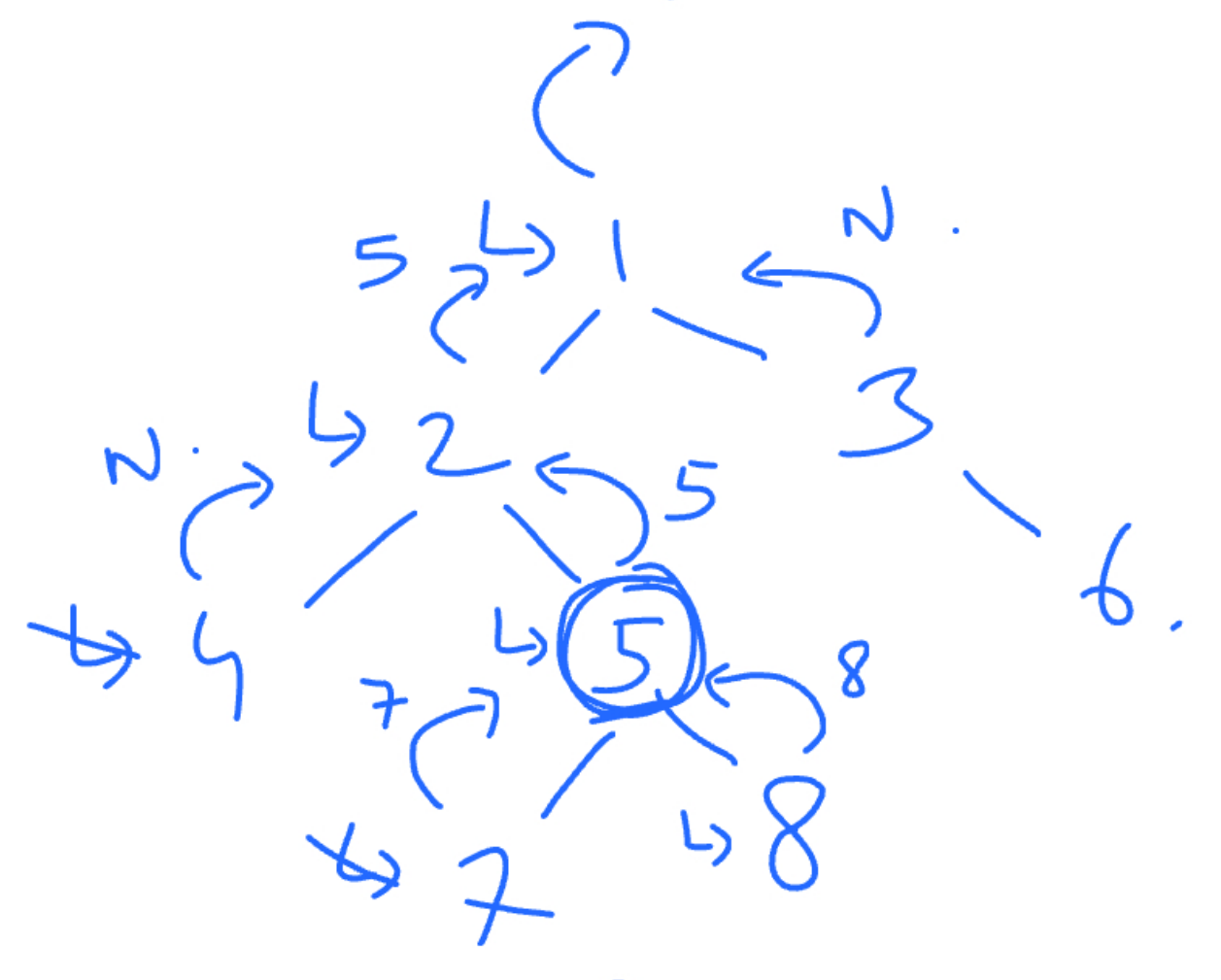
res = ~~1~~ 3



7, 8 → 5.

5, 2 → 2

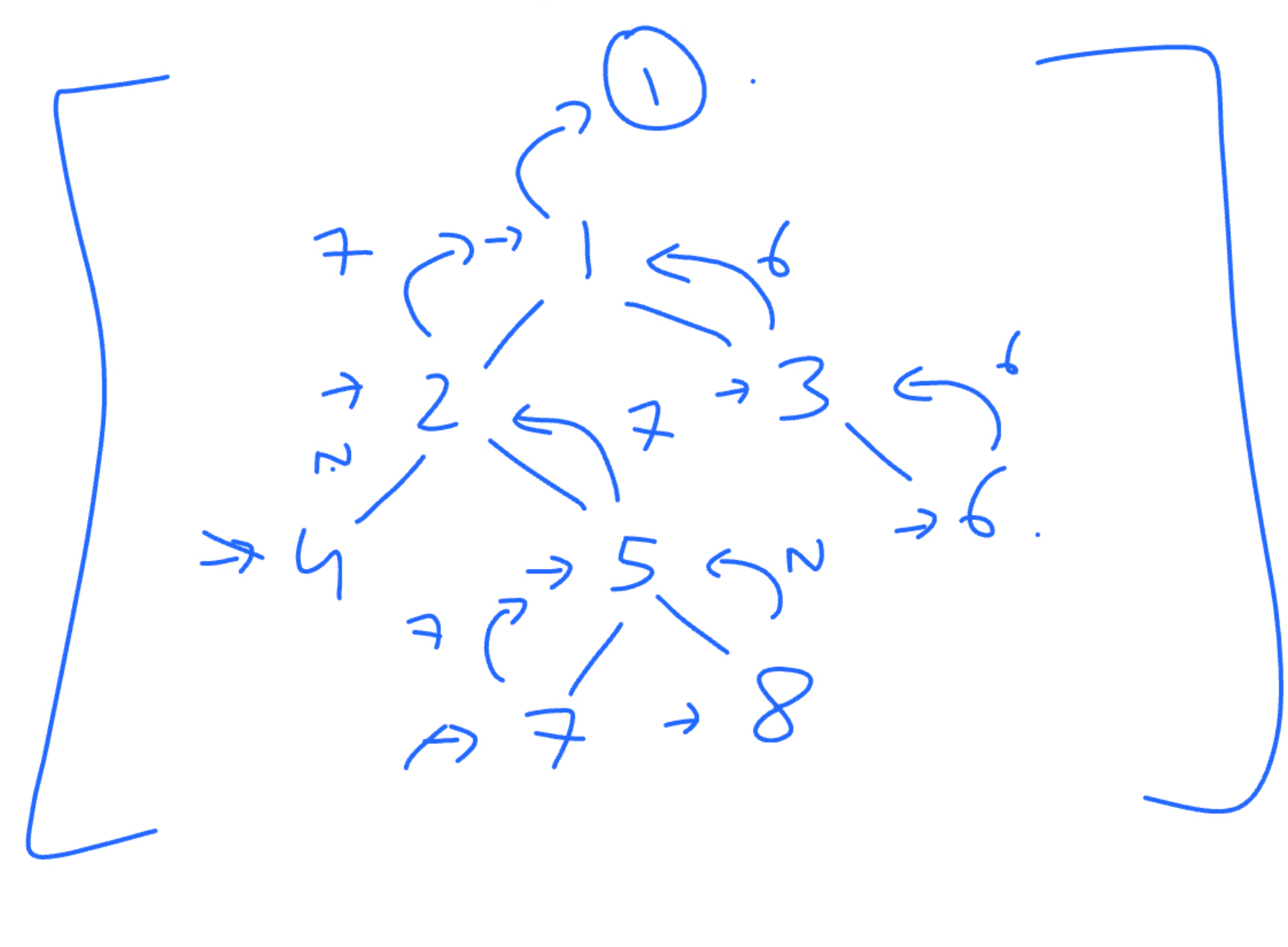
4, 6 → 1



5.
5
1
N.
N.
2
5
4
7
5
8
7
8
3
6.

n1 = 7

n2 = 8.



1
7
1
6
2
7
3
N
N
4
5
N
6.
7
8

n1 = 7

n2 = 6.

Node* lca (root, n1, n2) {

}

B.T using Inorder & Postorder.

in[] = [4, 2, 5, 1, 3].

post[] = [4, 5, 2, 3, 1].

Imp: Only a single unique B.T. can be built out of:

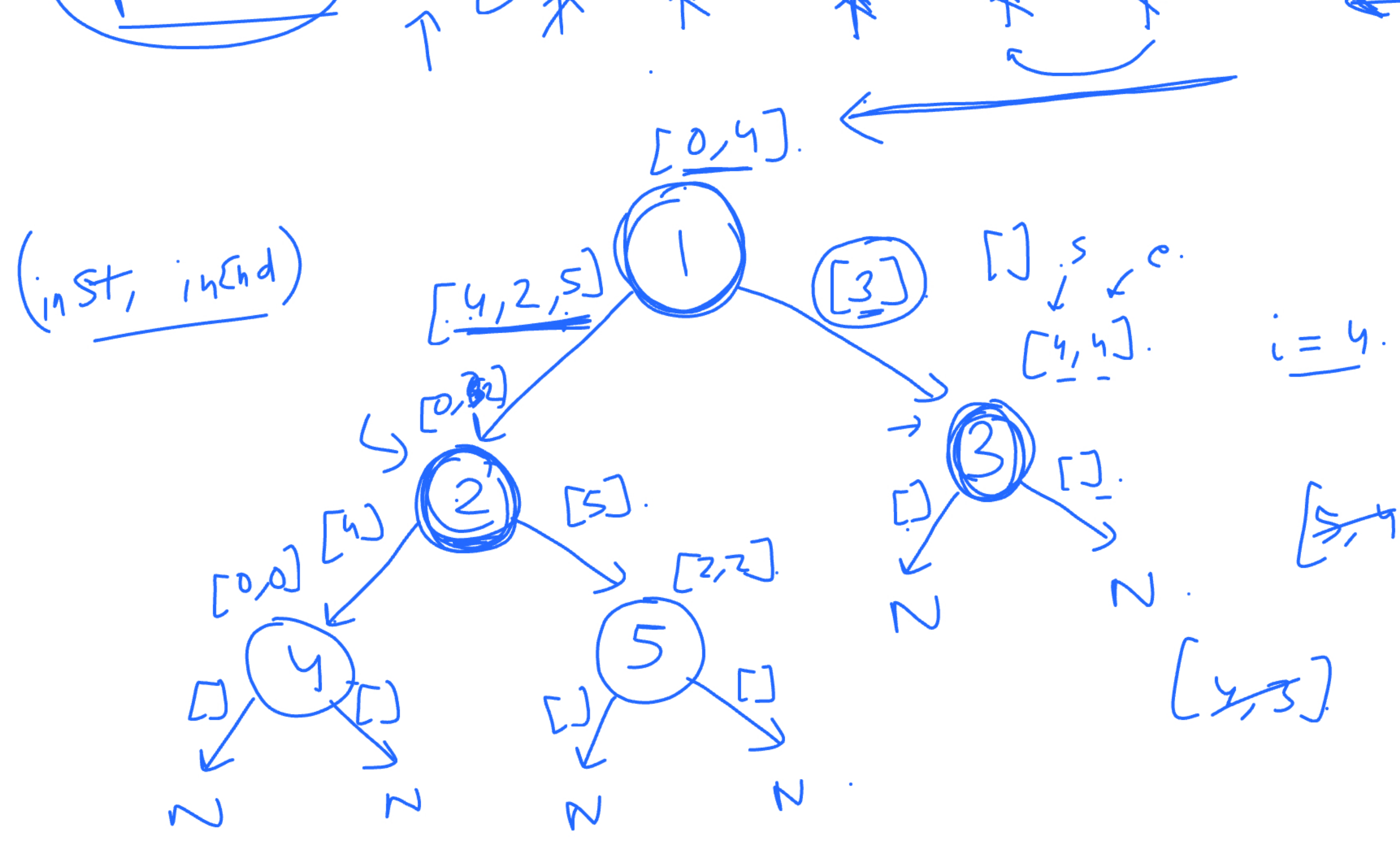
1) Inorder & Postorder
2) Inorder & Preorder.

---

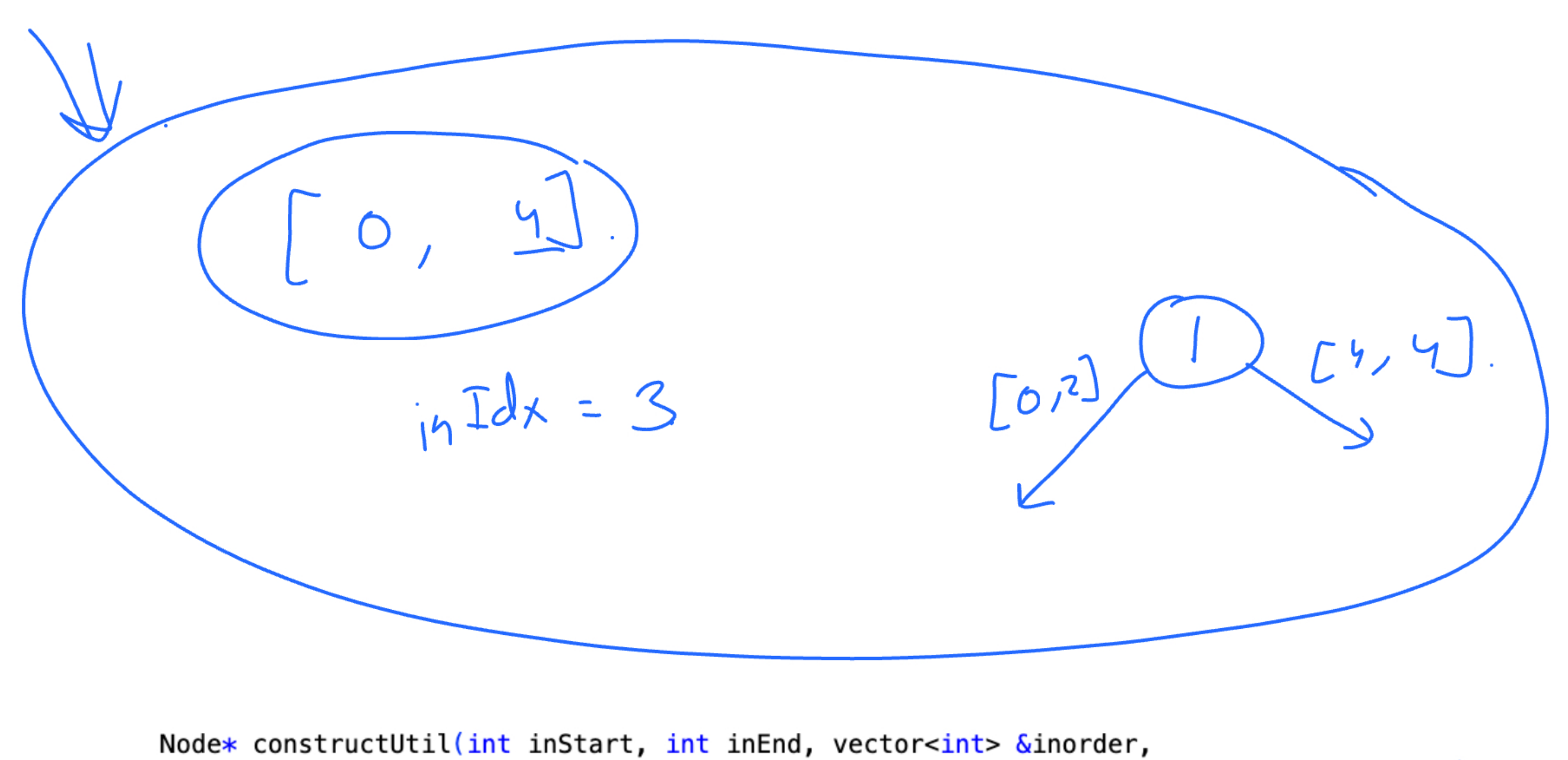
0 1 2 3 4

in[] = [4, 2, 5, 1, 3]. LSR.

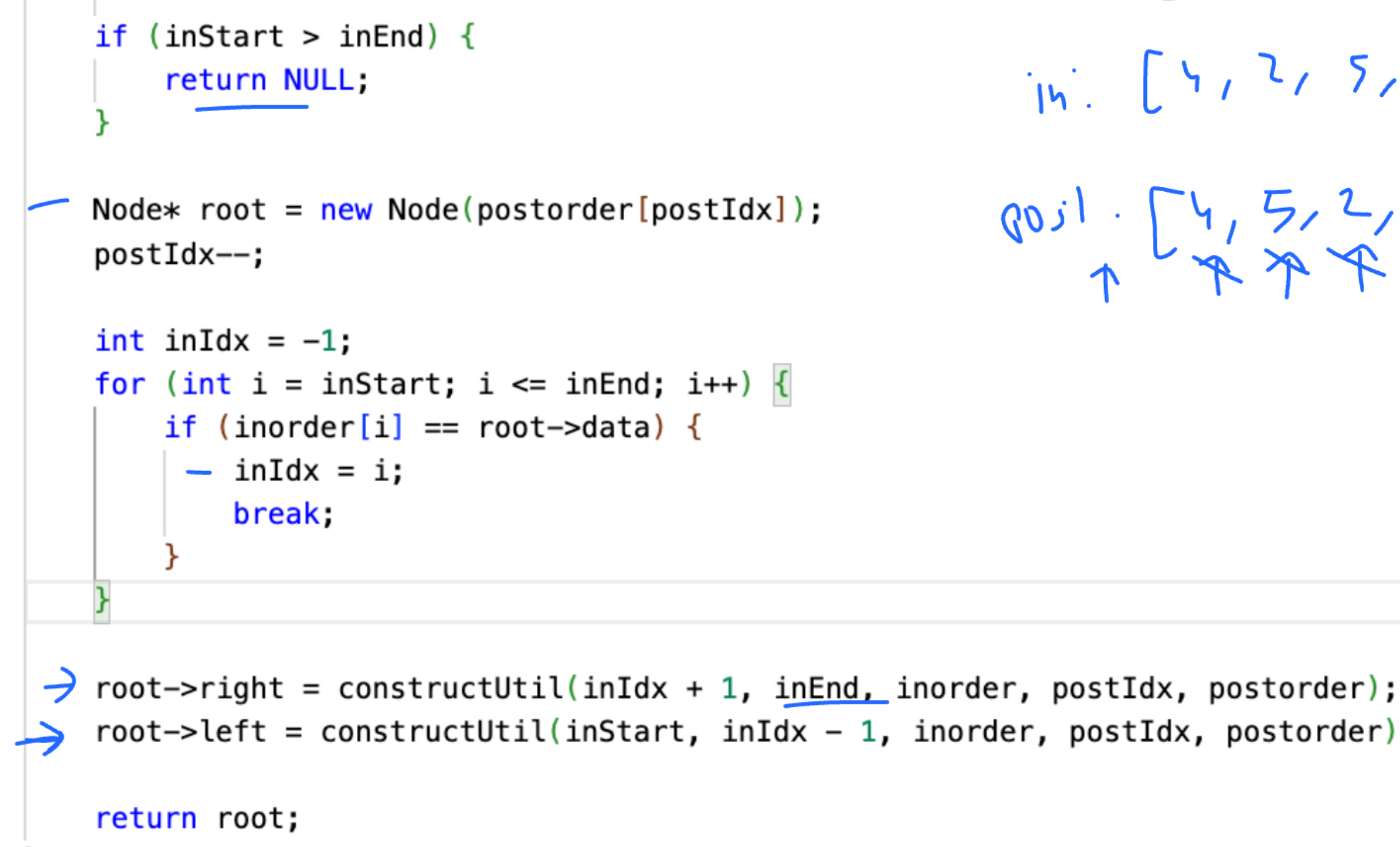
post[] = [4, 5, 2, 3, 1]. LRS.



(inSt, inEnd)
[0,4]
1
[4,2,5]
[3]
[0,2]
2
[4]
[5]
[0,0]
4
[]
[]
N
N
5
[2,2]
[]
[]
N
N
[], s, e
[4,4].
i = 4.
3
[]
[]
N
N.
~~[5,4]~~
[4,3]



[0, 4].
inIdx = 3
1
[0,2]
[4, 4].



```
Node* constructUtil(int inStart, int inEnd, vector<int> &inorder,
                    int &postIdx, vector<int> &postorder) {

    if (inStart > inEnd) {
        return NULL;
    }

    Node* root = new Node(postorder[postIdx]);
    postIdx--;

    int inIdx = -1;
    for (int i = inStart; i <= inEnd; i++) {
        if (inorder[i] == root->data) {
            inIdx = i;
            break;
        }
    }

    root->right = constructUtil(inIdx + 1, inEnd, inorder, postIdx, postorder);
    root->left = constructUtil(inStart, inIdx - 1, inorder, postIdx, postorder);

    return root;
}
```

0 1 2 3 4

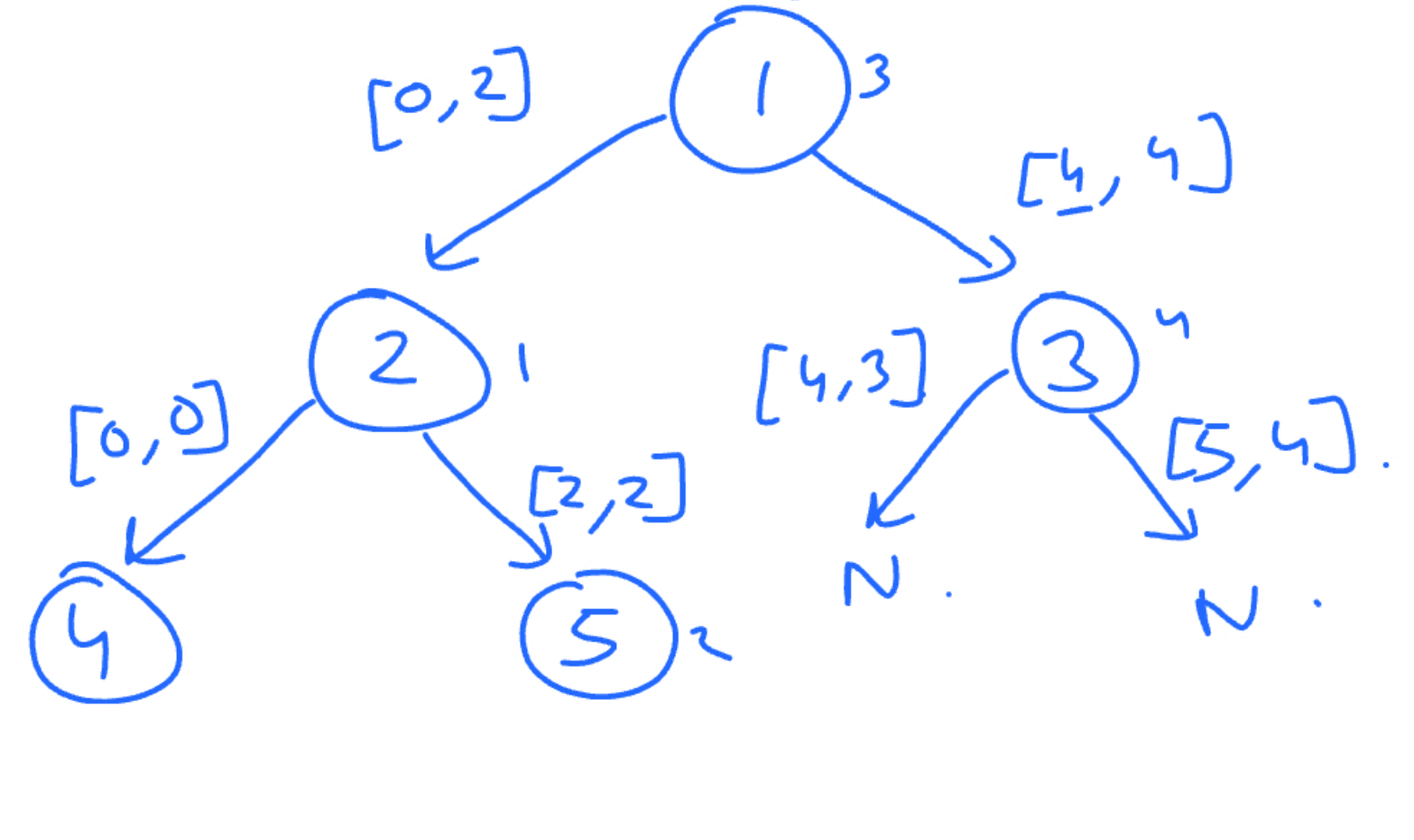
in: [4, 2, 5, 1, 3]

post: [4, 5, 2, 3, 1]

~~s = 0 4 5~~

~~e = 4 4 4~~

inIdx = ~~3 4~~

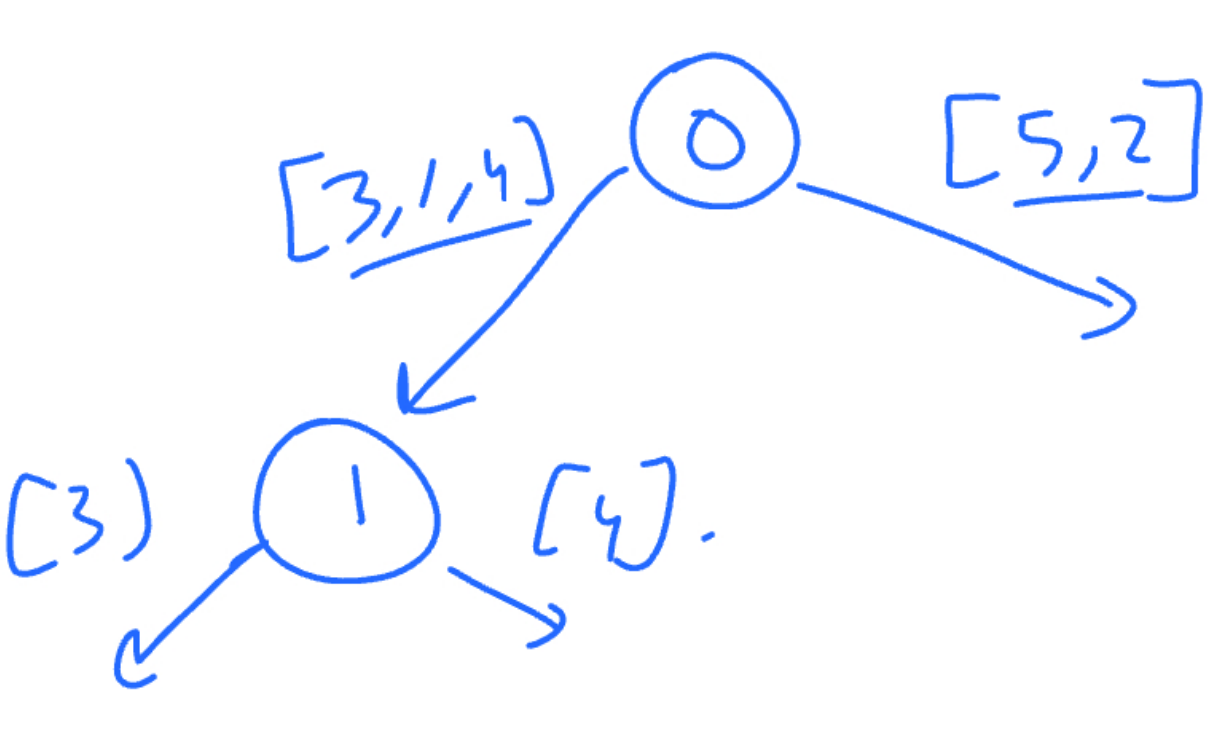


[0,4]
1
3
[0,2]
[4,4]
2
1
3
4
[0,0]
[2,2]
[4,3]
[5,4].
4
5
2
N.
N.

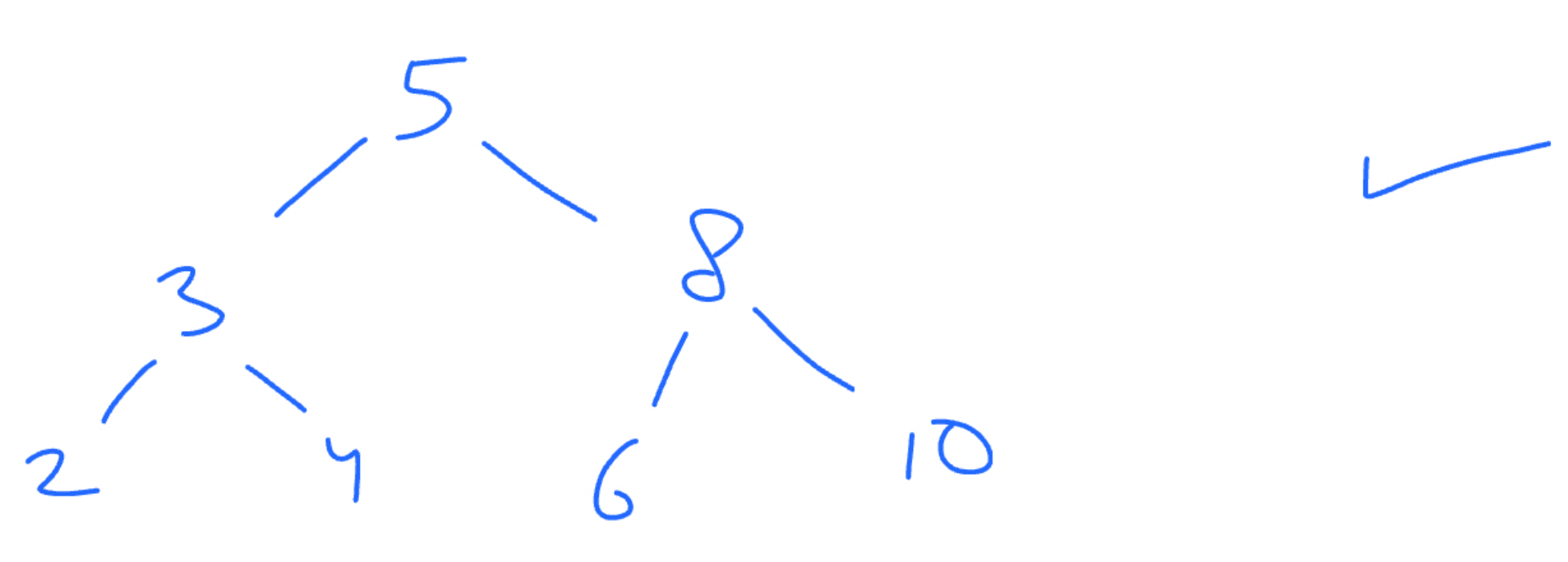
---

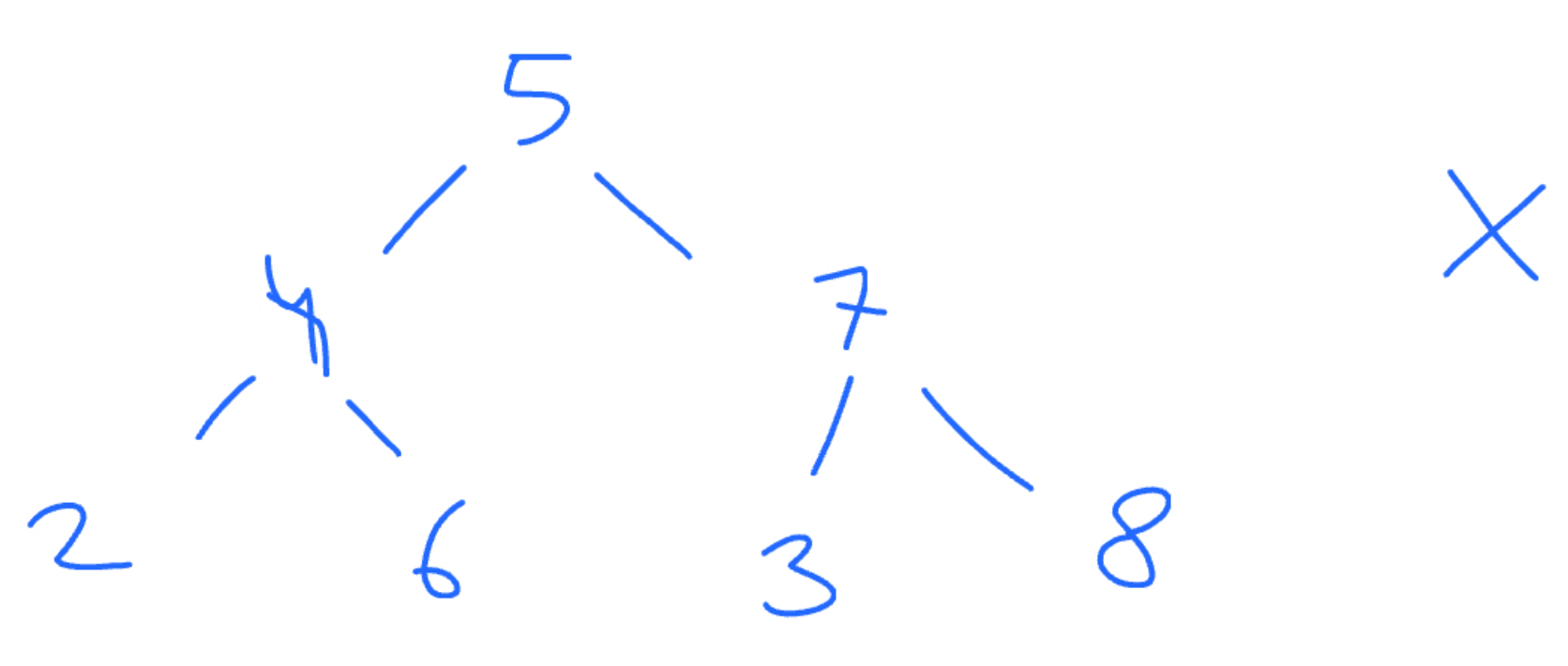
in → [3, 1, 4, 0, 5, 2]   LSR

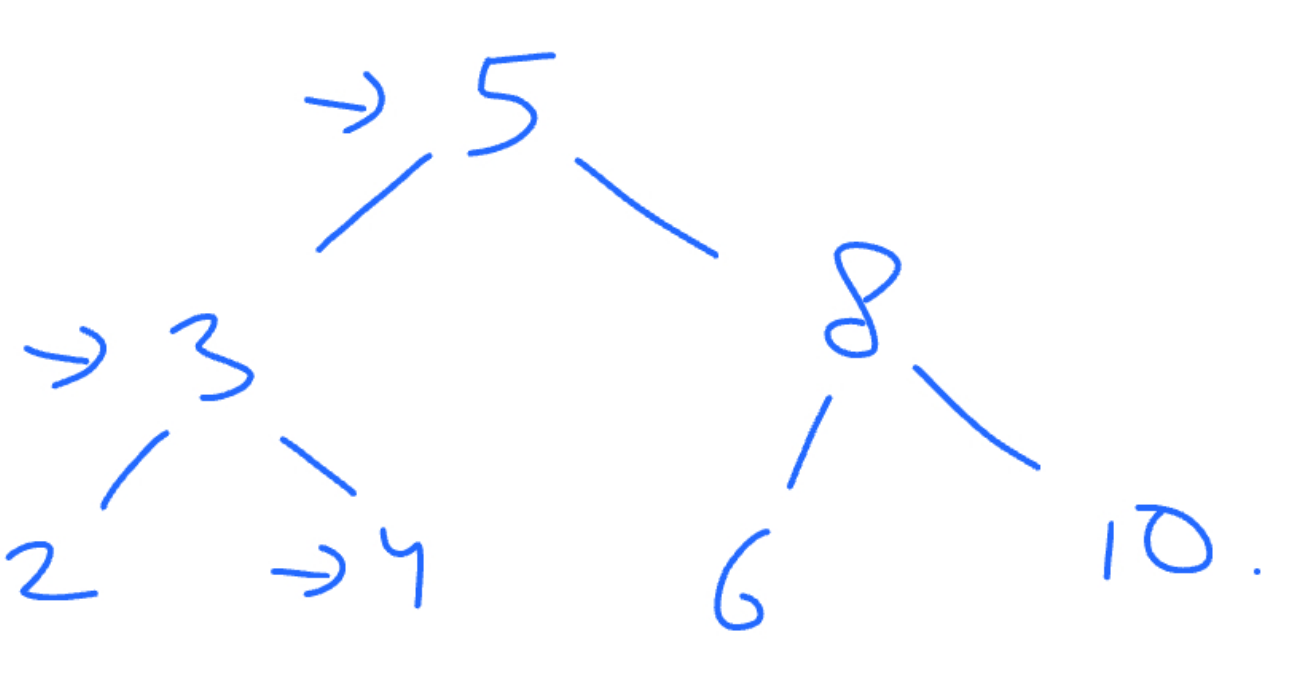
pre → [0, 1, 3, 4, 2, 5].   SLR



BST.







LSR

inorder: [2, 3, 4, 5, 6, 8, 10]

NOTE: Inorder trav. of a BST is always sorted.